Foundational Theory of Big Data and Processing Frameworks

The 3Vs of Big Data

Big Data is commonly defined through the **3Vs: Volume, Velocity, and Variety**, which capture the scale, speed, and complexity of modern data.

1. **Volume** – Refers to the enormous amount of data generated every second. Organizations collect terabytes or even petabytes of data from multiple sources.
   Example: Facebook generates over 4 petabytes of data per day from posts, images, and interactions. Retail giants like Walmart process transaction logs from millions of customers daily.
2. **Velocity** – Indicates the speed at which data is created, ingested, and processed. Businesses often need to handle real-time or near real-time data streams.
   Example: Stock market trading systems generate high-frequency data streams where price fluctuations must be analyzed instantly. IoT sensors in smart cities collect continuous streams of traffic, pollution, and weather data that require rapid decision-making.
3. **Variety** – Represents the different forms of data: structured, semi-structured, and unstructured. Traditional databases mainly handled structured rows and columns, but today's data includes multimedia and free-text formats.
   Example: A YouTube video (unstructured), Twitter tweets with hashtags and emojis (semi-structured), and customer transaction tables (structured) together form diverse datasets requiring specialized tools for integration and analysis.

Together, the 3Vs illustrate the challenges in storing, processing, and extracting insights from Big Data.

Hadoop MapReduce vs. Apache Spark

1. Architecture

- **Hadoop MapReduce**: A disk-based framework that processes data in two stages—Map (split and filter) and Reduce (aggregate). Each intermediate result is written to disk (HDFS), making it reliable but slower.
- **Apache Spark**: An in-memory distributed processing framework. It introduces the concept of **Resilient Distributed Datasets (RDDs)**, which store data in memory and only spill to disk when needed. This significantly reduces read/write overhead.

2. Performance

- **MapReduce**: High fault-tolerance but slower due to repeated disk I/O between every map and reduce stage.
- **Spark**: Faster (up to 100x in-memory, 10x on disk) because it keeps iterative computations in memory. This makes it ideal for algorithms requiring multiple passes over the data, such as machine learning or graph analytics.

3. Use Cases

- **MapReduce**: Best for batch processing on very large static datasets (e.g., nightly ETL jobs, log analysis, large-scale indexing).
- **Spark**: Suitable for diverse workloads—batch, streaming, interactive queries, and iterative machine learning. Examples include fraud detection (real-time), recommendation engines, and genomic research.

Why Spark is faster for iterative processing?

Iterative algorithms like PageRank or k-means clustering require multiple passes over the same dataset. In MapReduce, each iteration writes results back to HDFS and reloads them for the next step. Spark, by contrast, caches the dataset in memory (RDDs or DataFrames) and reuses it across iterations, drastically reducing overhead.

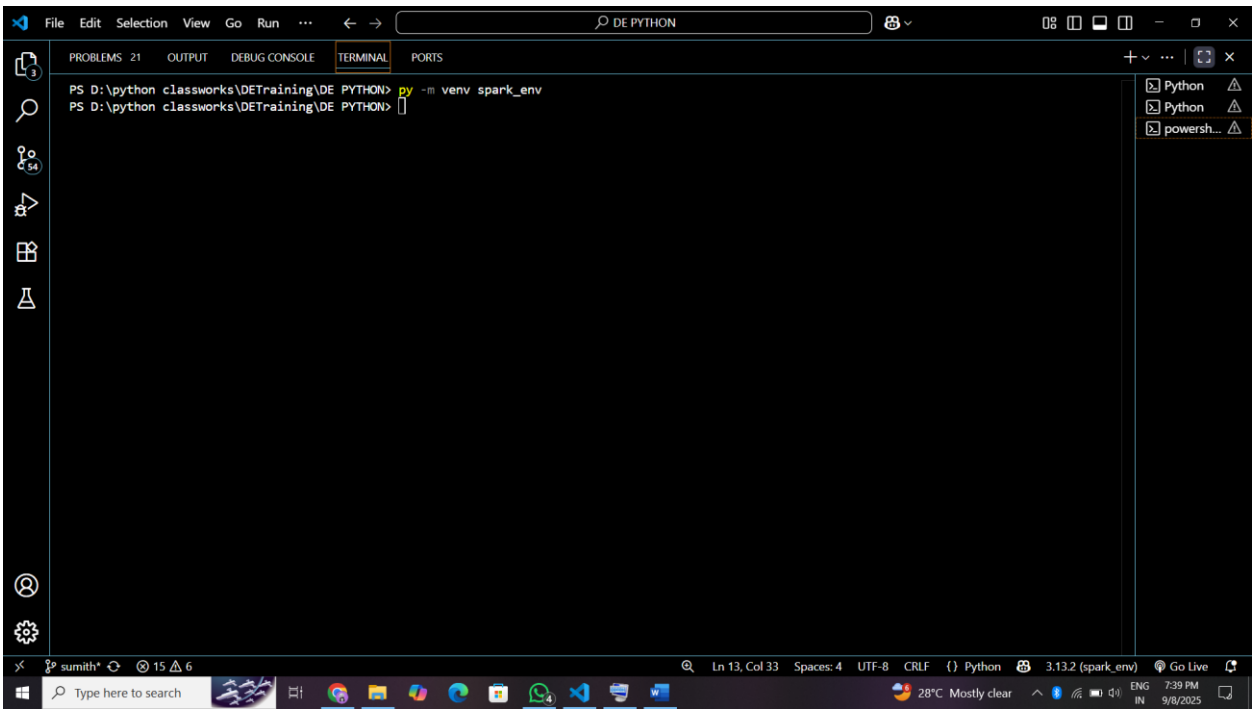
Local Development Setup for Apache Spark

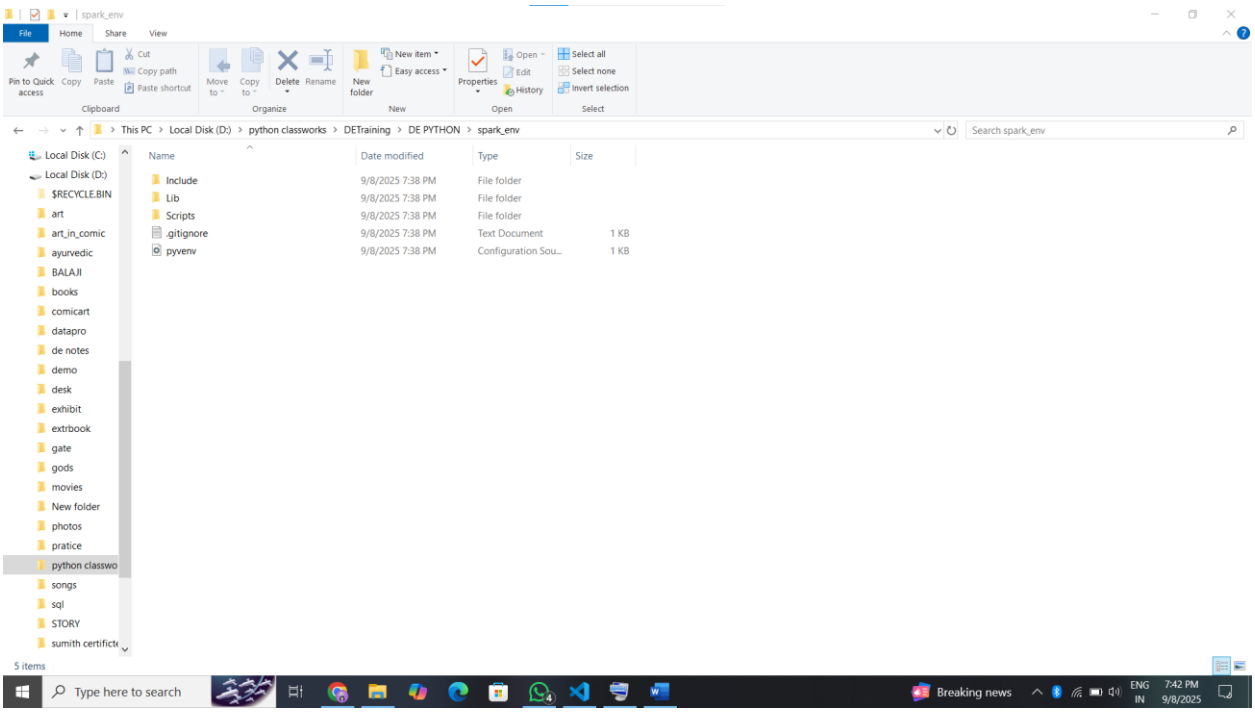
1. Create a Virtual Environment

Open your terminal (or VS Code/PyCharm terminal):

Create virtual environment

python -m venv spark_env





Activate the environment

On Windows:

spark_env\Scripts\activate

On Mac/Linux:

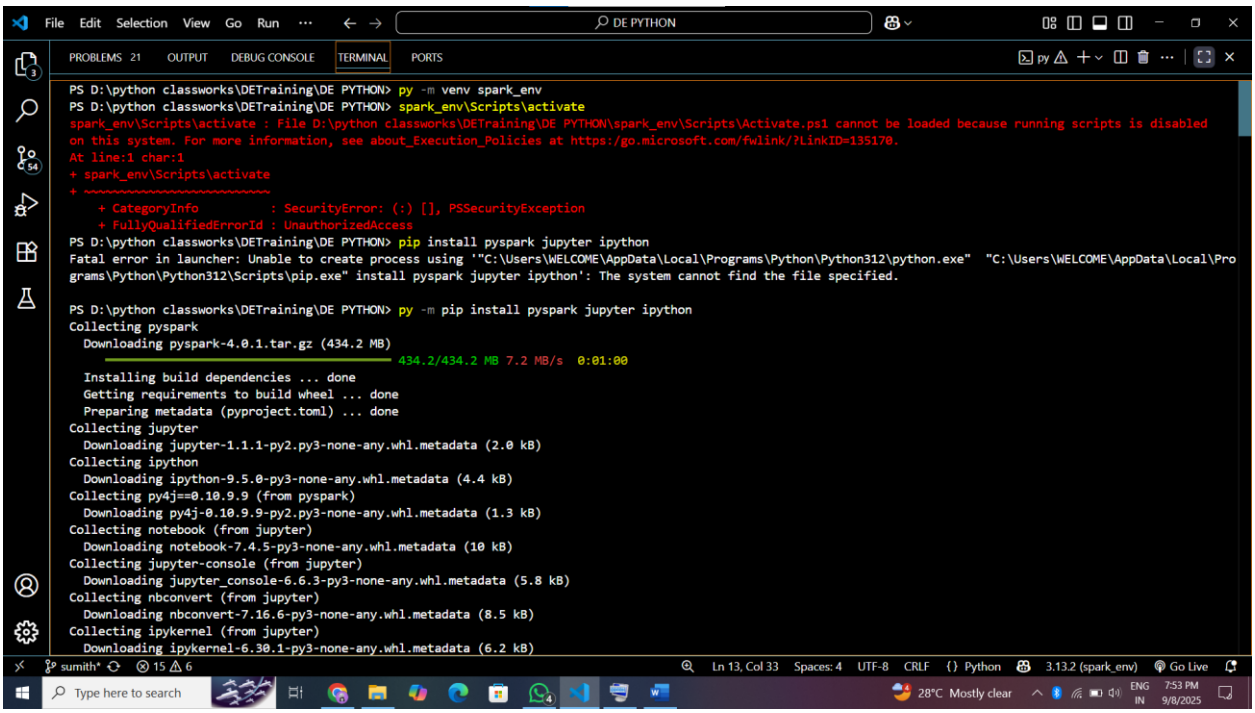
source spark_env/bin/activate

---

2. Install Required Packages

With the environment active, install the necessary libraries:

py -m pip install pyspark jupyter ipython



```
  Downloading types_python_dateutil-2.9.0.20250822-py3-none-any.whl.metadata (1.8 kB)
Collecting executing>=1.2.0 (from stack_data->ipython)
  Downloading executing-2.2.1-py2.py3-none-any.whl.metadata (8.9 kB)
Collecting asttokens>=2.1.0 (from stack_data->ipython)
  Downloading asttokens-3.0.0-py3-none-any.whl.metadata (4.7 kB)
Collecting pure-eval (from stack_data->ipython)
  Downloading pure_eval-0.2.3-py3-none-any.whl.metadata (6.3 kB)
Downloading py4j-0.10.9.9-py2.py3-none-any.whl (203 kB)
Downloading jupyter-1.1.1-py2.py3-none-any.whl (2.7 kB)
Downloading ipython-9.5.0-py3-none-any.whl (612 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 612.4/612.4 kB 6.3 MB/s  0:00:00
Downloading prompt_toolkit-3.0.52-py3-none-any.whl (391 kB)
Downloading jedi-0.19.2-py2.py3-none-any.whl (1.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 5.5 MB/s  0:00:00
Downloading parso-0.8.5-py2.py3-none-any.whl (106 kB)
Downloading pygments-2.19.2-py3-none-any.whl (1.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 6.3 MB/s  0:00:00
Downloading traitlets-5.14.3-py3-none-any.whl (85 kB)
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading decorator-5.2.1-py3-none-any.whl (9.2 kB)
Downloading ipykernel-6.30.1-py3-none-any.whl (117 kB)
Downloading comm-0.2.3-py3-none-any.whl (7.3 kB)
Downloading debugpy-1.8.16-cp313-cp313-win_amd64.whl (5.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.3/5.3 MB 7.2 MB/s  0:00:00
Downloading jupyter_client-8.6.3-py3-none-any.whl (106 kB)
Downloading jupyter_core-5.8.1-py3-none-any.whl (28 kB)
Downloading matplotlib_inline-0.1.7-py3-none-any.whl (9.9 kB)
Downloading nest_asyncio-1.6.0-py3-none-any.whl (5.2 kB)
Downloading platformdirs-4.4.0-py3-none-any.whl (18 kB)
Downloading psutil-7.0.0-cp37-abi3-win_amd64.whl (244 kB)
Downloading pywin32-311-cp313-cp313-win_amd64.whl (9.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.5/9.5 MB 5.4 MB/s  0:00:01
Downloading pyzmq-27.0.2-cp312-abi3-win_amd64.whl (619 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 619.3/619.3 kB 3.7 MB/s  0:00:00
```

Verify installations:

pip show pyspark

pip show jupyter

```
PS D:\python classworks\DETraining\DE PYTHON> & "D:/python classworks/DETraining/DE PYTHON/spark_env/Scripts/Activate.ps1"
& : File D:\python classworks\DETraining\DE PYTHON\spark_env\Scripts\Activate.ps1 cannot be loaded because running scripts is disabled on this system. For more
information, see about_Execution_Policies at https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:3
+ & "D:/python classworks/DETraining/DE PYTHON/spark_env/Scripts/Activa ...
+   ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
PS D:\python classworks\DETraining\DE PYTHON> py -m pip show pyspark
Name: pyspark
Version: 4.0.1
Summary: Apache Spark Python API
Home-page: https://github.com/apache/spark/tree/master/python
Author: Spark Developers
Author-email: dev@spark.apache.org
License: http://www.apache.org/licenses/LICENSE-2.0
Location: C:\Users\WELCOME\AppData\Local\Programs\Python\Python313\Lib\site-packages
Requires: py4j
Required-by:
PS D:\python classworks\DETraining\DE PYTHON> py -m pip show ipython
Name: ipython
Version: 9.5.0
Summary: IPython: Productive Interactive Computing
Home-page: https://ipython.org
Author: The IPython Development Team
Author-email: ipython-dev@python.org
License: BSD-3-Clause
Location: C:\Users\WELCOME\AppData\Local\Programs\Python\Python313\Lib\site-packages
Requires: colorama, decorator, ipython-pygments-lexers, jedi, matplotlib-inline, prompt_toolkit, pygments, stack_data, traitlets
Required-by: ipykernel, ipywidgets, jupyter-console
PS D:\python classworks\DETraining\DE PYTHON>
```

3. Validate Spark Installation with a Word Count Example

(a) Create a sample text file

```
echo "Apache Spark makes big data processing faster. Spark is amazing!" > sample.txt
```

(b) Write a simple wordcount.py script

```
from pyspark import SparkContext

# Initialize SparkContext
sc = SparkContext("local", "WordCountApp")

# Load the text file
text_file = sc.textFile("sample.txt")

# Perform word count
counts = (text_file
        .flatMap(lambda line: line.split(" "))
        .map(lambda word: (word, 1))
        .reduceByKey(lambda a, b: a + b))

# Collect results
for word, count in counts.collect():
    print(f"{word}: {count}")

sc.stop()
```

(c) Run the script

python wordcount.py

✅ Expected output (words with their counts):

Apache: 1

Spark: 2

makes: 1

big: 1

data: 1

processing: 1

faster.: 1

is: 1

amazing!: 1

---

4. Run Spark in Jupyter Notebook (Optional)

If you prefer interactive testing:

jupyter notebook

Inside a notebook cell:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("TestApp").getOrCreate()
df = spark.read.text("sample.txt")
df.show()
spark.stop()
```

*** NOTE: Didn’t executed because of error in activating the scripts.

1. Key Components of the Hadoop Ecosystem

Here's a **diagram-style explanation** of how the core Hadoop tools interact:

```
          +-------------------+
          |   Applications    |
          |  (Hive, Pig, etc) |
          +---------+---------+
                    |
                    v
           +--------+--------+
           |   YARN (Resource|
           |  Manager + Node)|
           +--------+--------+
                     |
       +-------------------+----------------------+
       |                                          |
       v                                          v
  +-------------------+              +-------------------+
|   HDFS (Storage)  |         |   Compute Engines |
|  (Blocks on Data  |         | (MapReduce, Spark)|
|   Nodes)          |         |                         |
 +-------------------+              +-------------------+
    ^                                   ^
    |                                   |
+-----+-----+                      +-------+-------+
| Sqoop     |                    | HBase (NoSQL) |
| (Import/  |                    |                     |
| Export)   |                    |                     |
+-----------+                      +---------------+
```

Key components:

- **HDFS**: Distributed storage system storing large datasets across nodes.
- **YARN**: Resource manager scheduling tasks across cluster nodes.
- **Hive**: SQL-like query engine for structured data.
- **HBase**: NoSQL database for real-time read/write access.
- **Sqoop**: Connects Hadoop with relational databases (imports/exports).

---

2. AWS EMR Cost Structure

Cost Elements:

1. **EC2 Instance Costs**: You pay for the EC2 instances (Master, Core, Task nodes). Pricing depends on instance type (e.g., m5.xlarge, r5.2xlarge), on-demand vs. spot.
2. **EMR Software Cost**: AWS charges **per instance per second** for EMR on top of EC2 cost. Typically ~$0.015–$0.03 per hour per instance (depends on region & version).
3. **Other Costs**: S3 storage, EBS volumes, data transfer.

Comparison:

- EC2 instances are usually the **biggest cost driver** (compute).
- EMR software adds a **small incremental fee** per instance.

Best Practices for Cost Optimization:

1. **Use Spot Instances** for task nodes (up to 70–80% cheaper than On-Demand).
2. **Enable Auto-termination** of idle clusters to avoid paying for unused resources.
3. **Use S3 for Storage** instead of HDFS (decouples storage from compute, cheaper long-term).

---

3. Spark Data Processing Task (Local Setup)

Assume you already have a CSV file sample_data.csv with columns like:

```
id,category,value
1,A,100
2,B,200
3,A,300
4,B,150
5,C,400
```

(a) Python script: process_csv.py

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum as _sum, avg

# Initialize Spark session
spark = SparkSession.builder.appName("CSVProcessingApp").getOrCreate()

# Read CSV into DataFrame
df = spark.read.csv("sample_data.csv", header=True, inferSchema=True)

print("Input Data:")
df.show()

# Filter rows where value > 150
filtered_df = df.filter(col("value") > 150)

print("Filtered Data (value > 150):")
```

```
filtered_df.show()

# Group by category and calculate sum & average of value
agg_df = (filtered_df
        .groupBy("category")
        .agg(_sum("value").alias("total_value"),
            avg("value").alias("avg_value")))

print("Aggregated Results:")
agg_df.show()

# Write results to Parquet
agg_df.write.mode("overwrite").parquet("output_parquet")

print("Results written to output_parquet/")

spark.stop()
```

(b) Run the script in VS Code terminal

```
python process_csv.py
```

(c) Expected Output

```
Filtered data (value > 150):
+---+--------+-----+
| id|category|value|
+---+--------+-----+
|  2|       B|  200|
|  3|       A|  300|
|  5|       C|  400|
+---+--------+-----+
Aggregated results:
+--------+-----------+------------------+
|category|total_value|         avg_value|
+--------+-----------+------------------+
|       B|        200|             200.0|
|       A|        300|             300.0|
|       C|        400|             400.0|
+--------+-----------+------------------+
Parquet results saved in output_parquet/.
```